PROGRAMAÇÃO ORIENTADA A OBJETOS

Kotlin
Classes, objetos e interfaces

Código explicado e experiências no Kotlin Playground

Fernando Lucatelli Nunes

2026/2027

O percurso da aula

A sequência principal contém os seis temas pedidos. O apêndice aprofunda a sintaxe encontrada nos projetos.

Preparação: ler um programa, distinguir tipos, declarar funções e seguir a execução.

Classes: construir objetos, guardar propriedades, definir métodos e proteger alterações.

Data classes e enums: representar valores e escolher entre alternativas conhecidas.

Object e companion object: usar um objeto partilhado e organizar a construção de instâncias.

Interfaces: definir operações comuns e executar implementações diferentes. No final, uma calculadora modular reúne os conceitos.

Um exemplo, um ficheiro, uma execução

Cada exemplo de código identifica um programa pelo seu ficheiro .kt. O ficheiro contém todas as declarações e uma única função main.

Quando um programa ocupa vários slides, o título ou a explicação assinala a parte apresentada. Para executar, copie o ficheiro completo.

A saída esperada aparece junto do exemplo e num ficheiro .out. Compare conteúdo e ordem das linhas.

Os exercícios têm ficheiros iniciais em exercicios/ e respostas em resolucoes/. A resolução surge num slide posterior.

Kotlin Playground

Abra https://play.kotlinlang.org e escolha Kotlin/JVM. Use uma versão estável atual.

Apague o exemplo inicial e cole o conteúdo completo de um único ficheiro .kt do pacote.

Prima Run. Leia a saída e compare-a com a saída esperada. Altere um dado e volte a executar.

Os exemplos desta aula usam dados escritos no programa. Pode experimentá-los sem instalar ferramentas nem fornecer entrada pela consola.

TODO("mensagem") compila, mas interrompe a execução quando é chamado. Nos exercícios, substitua-o pela implementação pedida.

A primeira leitura de Kotlin

CÓDIGO programas/P04_PrimeiroPrograma.kt

```kotlin
fun main() {
    // Esta linha é um comentário.
    val nome: String = "Ana"
    println(nome)
    println("Biblioteca")
}
```

SAÍDA

```
Ana
Biblioteca
```

fun declara uma função. main é o ponto de entrada: a execução começa no seu corpo.

Os parênteses vazios indicam que esta main não recebe parâmetros. As chavetas delimitam o corpo.

val declara nome. Os dois pontos indicam o tipo String. = associa à variável o texto entre aspas.

println(...) chama uma função com um argumento e termina a linha. As instruções executam-se de cima para baixo.

// inicia um comentário até ao fim da linha. A indentação ajuda a leitura. Não é necessário terminar cada instrução com ;.

Variáveis e tipos

CÓDIGO programas/P05_Variaveis.kt

```kotlin
fun main() {
    val limite: Int = 3
    var quantidade: Int = 1
    quantidade = quantidade + 1
    val permitido: Boolean = quantidade <= limite
    val grande: Long = 12L
    println(quantidade)
    println(permitido)
    println(grande)
}
```

SAÍDA

```
2
true
12
```

val impede reatribuir a variável depois de inicializada. var permite uma nova atribuição.

Int guarda inteiros de 32 bits. Long guarda inteiros de 64 bits. O sufixo L explicita um literal Long.

Boolean admite true e false. <= compara os números e produz um Boolean.

Na atribuição quantidade = quantidade + 1, calcula-se primeiro o lado direito com o valor antigo.

O tipo pode ser inferido: val limite = 3 também declara um Int. "3" seria texto e teria tipo String.

Expressões e aritmética inteira

CÓDIGO programas/P06_Aritmetica.kt

```
01 fun main() {
02     val total = 95
03     val horas = total / 60
04     val minutos = total % 60
05     println(horas)
06     println(minutos)
07     println(horas * 60 + minutos)
08 }
```

SAÍDA

```
1
35
95
```

/ entre inteiros calcula o quociente inteiro. 95 / 60 vale 1. O resultado continua inteiro.

% calcula o resto. 95 % 60 vale 35 porque 95 = 1 × 60 + 35.

* multiplica, + soma e - subtrai. Multiplicação e divisão têm prioridade sobre adição e subtração.

Os parênteses agrupam uma expressão: (2 + 3) * 4 vale 20. O valor calculado pode ser guardado ou passado a uma função.

Em Kotlin, -1 % 5 vale -1. Para representantes modulares não negativos, faremos uma normalização explícita.

Condições e comparações

CÓDIGO programas/P07_Condicoes.kt

```
01 fun main() {
02     val hora = 23
03     val valida = hora >= 0 && hora <= 23
04     if (valida) {
05         println("Hora aceite")
06     } else {
07         println("Hora recusada")
08     }
09     println(hora == 23)
10     println(hora != 23)
11 }
```

SAÍDA

```
Hora aceite
true
false
```

if testa a condição entre parênteses. Quando é true, executa o primeiro bloco. Quando é false, executa o bloco else.

&& exige que ambas as condições sejam verdadeiras. || aceita pelo menos uma. ! inverte um Boolean.

&& e || avaliam o lado direito apenas quando é necessário para determinar o resultado.

== testa igualdade e != testa desigualdade. = serve para inicializar ou atribuir. <, >, <= e >= comparam ordem.

hora in 0..23 exprime o mesmo intervalo inclusivo. .. inclui os extremos e in testa pertença ao intervalo.

Funções, parâmetros e resultado

CÓDIGO programas/P08_Funcoes.kt

```kt
fun totalMinutos(h: Int, m: Int): Int {
    return h * 60 + m
}

fun main() {
    val total = totalMinutos(1, 35)
    println(total)
}
```

SAÍDA

```
95
```

Na declaração, h e m são parâmetros de tipo Int. A vírgula separa-os. : Int depois de) indica o tipo do resultado.

Na chamada totalMinutos(1, 35), os argumentos 1 e 35 ocupam as posições de h e m.

return entrega o valor calculado a quem chamou a função e termina esta execução da função.

A chamada calcula 1 * 60 + 35, devolve 95 e só depois inicializa total. A linha seguinte imprime 95.

Declarar a função descreve o trabalho. O corpo executa quando existe uma chamada. Funções podem existir diretamente num ficheiro.

Corpo de expressão e Unit

CÓDIGO programas/P09_ExpressaoUnit.kt

```kotlin
fun totalMinutos(h: Int, m: Int): Int = h * 60 + m

fun mostrar(total: Int): Unit {
    println(total)
}

fun main() {
    mostrar(totalMinutos(1, 35))
}
```

SAÍDA

```
95
```

= depois da assinatura introduz um corpo de expressão. Essa expressão fornece diretamente o resultado, sem return.

mostrar recebe um Int e apresenta-o. Unit indica que a função não fornece um resultado útil ao chamador.

Numa função com corpo entre chavetas, : Unit pode ser omitido. O corpo continua a executar normalmente.

Na última linha, a chamada interior termina primeiro. mostrar recebe então 95 e imprime-o.

Calcular um resultado permite reutilizá-lo noutra conta. println apresenta o valor ao utilizador.

Texto, interpolação e ponto

CÓDIGO programas/P10_Textos.kt

```kotlin
fun main() {
    val nome = " Ana "
    val limpo = nome.trim()
    val minutos = 35
    println("Nome: $limpo")
    println("Letras: ${limpo.length}")
    println("Total: ${60 + minutos}")
}
```

SAÍDA

```
Nome: Ana
Letras: 3
Total: 95
```

Uma String é um texto entre aspas. $limpo insere o valor da variável na mensagem.

${...} permite inserir o resultado de uma expressão, como limpo.length ou 60 + minutos.

O ponto permite consultar propriedades ou chamar funções sobre o valor à esquerda. nome.trim() devolve texto sem espaços exteriores.

length é uma propriedade consultada sem parênteses. O seu valor é o comprimento do texto.

trim() devolve outra String. A variável nome continua a referir o texto original. A seguir definiremos os nossos próprios tipos.

Uma classe descreve os objetos que podemos criar

CÓDIGO programas/C01_PrimeiroLivro.kt

```
01 class Livro(val id: String, val titulo: String)
02
03 fun main() {
04     val livro = Livro("L1", "Mensagem")
05     println(livro.id)
06     println(livro.titulo)
07 }
```

SAÍDA

```
L1
Mensagem
```

Uma classe define um tipo. Livro descreve os dados guardados em cada objeto desse tipo.

Os parênteses após Livro declaram o construtor primário. id e titulo recebem argumentos quando construímos o objeto.

val transforma esses parâmetros em propriedades de leitura. Cada objeto guarda os seus próprios valores.

Livro("L1", "Mensagem") cria uma instância. livro guarda uma referência a essa instância. Kotlin dispensa a palavra new.

O ponto seleciona uma propriedade: livro.id. A classe pode terminar sem chavetas quando não tem corpo.

Dois exemplares têm os seus próprios dados

CÓDIGO programas/C02_DoisLivros.kt

```kotlin
class Livro(val id: String, val titulo: String)

fun main() {
    val primeiro = Livro("L1", "Mensagem")
    val segundo = Livro("L2", "Mensagem")
    println("${primeiro.id}: ${primeiro.titulo}")
    println("${segundo.id}: ${segundo.titulo}")
}
```

SAÍDA

```
L1: Mensagem
L2: Mensagem
```

Cada chamada ao construtor cria um objeto. Usamos a mesma classe para representar duas cópias físicas do livro.

primeiro e segundo referem objetos diferentes. O título pode coincidir enquanto os identificadores são diferentes.

${primeiro.id} avalia o acesso à propriedade e insere o resultado no texto. As chavetas pertencem à interpolação.

O identificador do exemplar é um dado do nosso modelo. A classe, sozinha, ainda não impede identificadores repetidos.

Parâmetro do construtor e propriedade

CÓDIGO programas/C03_ParametroPropriedade.kt

```
01 class Rotulo(texto: String) {
02     val textoGuardado: String = texto
03 }
04
05 fun main() {
06     val rotulo = Rotulo("Biblioteca")
07     println(rotulo.textoGuardado)
08 }
```

SAÍDA

```
Biblioteca
```

No construtor, texto não tem val nem var. É um parâmetro usado durante a inicialização.

A declaração val textoGuardado cria uma propriedade e guarda nela o valor recebido em texto.

Fora da classe, rotulo.textoGuardado existe. rotulo.texto não existe, pois texto não foi declarado como propriedade.

Escrever class Rotulo(val texto: String) seria outra opção: criaria diretamente a propriedade texto.

Um método usa os dados do seu objeto

CÓDIGO programas/C04_MetodoDescricao.kt

```kotlin
class Livro(val id: String, val titulo: String) {
    fun descricao(): String {
        return "$id: $titulo"
    }
}

fun main() {
    val livro = Livro("L1", "Mensagem")
    val texto = livro.descricao()
    println(texto)
}
```

SAÍDA

```
L1: Mensagem
```

Uma função declarada dentro da classe é um método. descricao pertence aos objetos de Livro.

Os parênteses vazios indicam ausência de parâmetros. : String declara o tipo do resultado. return entrega esse resultado.

Dentro do método, id e titulo são as propriedades do objeto sobre o qual fizemos a chamada.

livro.descricao() executa o método. Guardar o resultado em texto não o mostra no ecrã: println faz essa apresentação.

this identifica o objeto que recebe a chamada

CÓDIGO programas/C05_This.kt

```
01 class Contador(var valor: Int) {
02     fun definir(valor: Int) {
03         this.valor = valor
04     }
05 }
06
07 fun main() {
08     val contador = Contador(2)
09     contador.definir(7)
10     println(contador.valor)
11 }
```

SAÍDA

```
7
```

O construtor declara uma propriedade var chamada valor. var permite atribuir-lhe outro inteiro depois da construção.

O método também recebe um parâmetro chamado valor. Dentro do método, o nome isolado refere esse parâmetro.

this.valor seleciona a propriedade do objeto atual. A atribuição guarda nessa propriedade o argumento recebido.

contador.definir(7) muda o estado desse contador. O método não declara resultado útil, pelo que o seu tipo de retorno é Unit.

Argumentos com nome e valores por omissão

CÓDIGO programas/C06_Argumentos.kt

```
01 class Leitor(
02     val nome: String,
03     val ativo: Boolean = true
04 )
05
06 fun main() {
07     val ana = Leitor("Ana")
08     val rui = Leitor(ativo = false, nome = "Rui")
09     println("${ana.nome}: ${ana.ativo}")
10     println("${rui.nome}: ${rui.ativo}")
11 }
```

SAÍDA

```
Ana: true
Rui: false
```

No construtor, = true define o valor usado quando a chamada omite o argumento ativo.

Leitor("Ana") fornece nome e usa ativo igual a true. Nenhum outro objeto interfere nessa escolha.

Na segunda chamada, ativo = false e nome = "Rui" são argumentos com nome. O nome identifica o parâmetro correspondente.

Os nomes permitem escrever os argumentos noutra ordem. A atribuição de um valor por omissão aparece na declaração, não na chamada.

init verifica os dados durante a construção

CÓDIGO programas/C07_InitRequire.kt

```
01 class Hora(val horas: Int, val minutos: Int) {
02     init {
03         require(horas >= 0 && horas <= 23) {
04             "A hora deve estar entre 0 e 23."
05         }
06     }
07 }
08
09 fun main() {
10     val partida = Hora(23, 40)
11     println(partida.horas)
12 }
```

SAÍDA

```
23
```

init delimita um bloco que executa durante a construção. Aqui verifica horas. No slide seguinte validaremos também minutos.

>= e <= comparam inteiros. && exige que ambas as condições sejam verdadeiras.

require recebe uma condição Boolean. Quando é falsa, lança IllegalArgumentException e interrompe a construção normal.

As chavetas após require contêm uma lambda sem parâmetros: uma pequena função que produz a mensagem quando a condição falha.

Troque 23 por 24 e execute de novo. A construção falha e println já não é executado.

Intervalos e regras completas para Hora

CÓDIGO programas/C08_Intervalos.kt

```
01 class Hora(val horas: Int, val minutos: Int) {
02     init {
03         require(horas in 0..23 && minutos in 0..59) {
04             "Hora inválida."
05         }
06     }
07 }
08
09 fun main() {
10     val partida = Hora(23, 40)
11     println("${partida.horas}:${partida.minutos}")
12 }
```

SAÍDA

```
23:40
```

0..23 cria um intervalo inclusivo: contém os inteiros de 0 até 23, incluindo ambos os extremos.

horas in 0..23 pergunta se horas pertence ao intervalo. O resultado é true ou false.

&& combina as regras de horas e minutos. Com 23 e 40, ambas são verdadeiras e a construção termina.

Com 24 e 0, a condição falha. Com 8 e 60, também falha. Os dois campos precisam de ser válidos.

Estas regras são invariantes do modelo: todo objeto Hora construído com sucesso deve respeitá-las.

private limita o acesso ao estado

CÓDIGO programas/C09_Private.kt

```
01 class Contador {
02     private var valor = 0
03     fun avancar() {
04         require(valor < 3) { "Limite atingido." }
05         valor += 1
06     }
07     fun atual(): Int = valor
08 }
09
10 fun main() {
11     val contador = Contador()
12     contador.avancar()
13     println(contador.atual())
14 }
```

SAÍDA

```
1
```

private permite aceder a valor apenas dentro da classe. O código de main não pode escrever contador.valor.

avancar e atual são públicos por omissão. Formam as operações que o resto do programa pode usar.

valor += 1 soma um e guarda o resultado em valor. Aqui os valores ficam limitados ao intervalo de 0 a 3.

fun atual(): Int = valor tem corpo de expressão: o resultado é o valor da expressão escrita depois de =.

O encapsulamento reúne estado e operações, controlando o acesso para que cada alteração respeite as regras estabelecidas.

private set permite consultar e controla alterações

CÓDIGO programas/C10_PrivateSet.kt

```
01  class Emprestimo {
02      var ativo: Boolean = true
03          private set
04      fun devolver() {
05          require(ativo) { "Já foi devolvido." }
06          ativo = false
07      }
08  }
09
10  fun main() {
11      val emprestimo = Emprestimo()
12      println(emprestimo.ativo)
13      emprestimo.devolver()
14      println(emprestimo.ativo)
15  }
```

SAÍDA

```
true
false
```

A propriedade ativo tem um getter para leitura e um setter para atribuição. Kotlin fornece estes acessores automaticamente.

private set restringe apenas o setter. Fora da classe, emprestimo.ativo continua disponível para consulta.

Fora da classe, emprestimo.ativo = false não compila. A alteração passa pelo método devolver.

require(ativo) aceita a devolução apenas enquanto o empréstimo está ativo. Uma segunda chamada é recusada antes da atribuição.

Uma propriedade também pode calcular o seu valor

CÓDIGO programas/C11_Getter.kt

```
01 class Emprestimo {
02     var ativo = true
03         private set
04     val devolvido: Boolean
05         get() = !ativo
06     fun devolver() {
07         require(ativo) { "Já foi devolvido." }
08         ativo = false
09     }
10 }
11 fun main() {
12     val emprestimo = Emprestimo()
13     println(emprestimo.devolvido)
14     emprestimo.devolver()
15     println(emprestimo.devolvido)
16 }
```

SAÍDA

```
false
true
```

get() define o código executado quando alguém lê devolvido. O = introduz o corpo de expressão desse getter.

! é a negação lógica: !true produz false e !false produz true.

devolvido calcula o resultado a partir de ativo. Neste exemplo, não guarda um segundo Boolean independente.

Embora devolvido seja val, leituras em momentos diferentes podem devolver valores diferentes, porque ativo pode mudar.

val protege a referência, não todo o objeto

CÓDIGO programas/C12_ValReferencia.kt

```kotlin
class Livro(val id: String, var titulo: String)

fun main() {
    val livro = Livro("L1", "Mensagen")
    livro.titulo = "Mensagem"
    println(livro.titulo)
}
```

SAÍDA

```
Mensagem
```

val livro impede atribuir outra referência à variável livro depois da sua inicialização.

O objeto referido pode ter propriedades mutáveis. titulo é var, pelo que livro.titulo = "Mensagem" é permitido.

id é val, pelo que uma atribuição externa a livro.id não é permitida.

A declaração da variável e as declarações das propriedades controlam coisas diferentes. val, sozinho, não garante imutabilidade profunda.

Duas variáveis podem referir o mesmo objeto

CÓDIGO programas/C13_Referencias.kt

```kotlin
class Livro(val id: String, var titulo: String)

fun main() {
    val primeiro = Livro("L1", "Mensagen")
    val outroNome = primeiro
    outroNome.titulo = "Mensagem"
    println(primeiro.titulo)
    println(outroNome.titulo)
}
```

SAÍDA

```
Mensagem
Mensagem
```

Apenas Livro("L1", "Mensagen") chama o construtor. O programa cria um objeto Livro.

val outroNome = primeiro copia a referência. As duas variáveis permitem chegar ao mesmo objeto.

A atribuição através de outroNome altera esse objeto. A leitura através de primeiro observa a alteração.

Para criar outro objeto, faça outra chamada ao construtor. Copiar uma referência não faz essa construção.

Devolver texto e apresentar texto

CÓDIGO programas/C14_RetornoApresentacao.kt

```kotlin
class Livro(val id: String, val titulo: String) {
    fun descricao(): String = "$id: $titulo"
    fun mostrar() {
        println(descricao())
    }
}

fun main() {
    val livro = Livro("L1", "Mensagem")
    val texto = livro.descricao()
    println("Ficha: $texto")
    livro.mostrar()
}
```

SAÍDA

```
Ficha: L1: Mensagem
L1: Mensagem
```

descricao devolve uma String. main pode guardá-la, acrescentar um prefixo ou usá-la noutra operação.

mostrar chama descricao no objeto atual e imprime o resultado. O seu tipo de retorno é Unit.

Em println(descricao()), Kotlin avalia primeiro descricao() e passa a String obtida como argumento de println.

Separar cálculo e apresentação permite reutilizar o mesmo método com a consola, um ficheiro ou outra forma de interação.

Playground: minutos desde o início do dia

EXERCÍCIO exercicios/C15_Hora.kt

```
01 class Hora(val horas: Int, val minutos: Int) {
02     init {
03         require(horas in 0..23 && minutos in 0..59) {
04             "Hora inválida."
05         }
06     }
07     fun totalMinutos(): Int = TODO("Calcular")
08 }
09 fun main() {
10     val hora = Hora(2, 30)
11     println(hora.totalMinutos())
12 }
```

SAÍDA DEPOIS DE COMPLETAR

```
150
```

Complete totalMinutos para devolver a quantidade de minutos decorridos desde as 00:00. Conserve o construtor e a validação.

Cada hora corresponde a 60 minutos. À contribuição das horas, acrescente os minutos indicados no segundo campo.

Substitua TODO("Calcular") pela expressão necessária. TODO é uma função que lança NotImplementedError quando a execução a alcança.

Com Hora(2, 30), o resultado deve ser 150. Teste também Hora(0, 0) e Hora(23, 59): espere 0 e 1439.

Por fim, experimente Hora(24, 0). A validação deve recusar a construção, antes de chamar totalMinutos.

Resolução: cada hora contribui com 60 minutos

RESOLUÇÃO resolucoes/C16_Hora.kt

```kotlin
class Hora(val horas: Int, val minutos: Int) {
    init {
        require(horas in 0..23 && minutos in 0..59) {
            "Hora inválida."
        }
    }
    fun totalMinutos(): Int = horas * 60 + minutos
}
fun main() {
    val hora = Hora(2, 30)
    println(hora.totalMinutos())
}
```

SAÍDA

```
150
```

horas * 60 converte a parte das horas em minutos. A multiplicação executa antes da soma.

Com os dados do exemplo, obtemos 2 * 60 + 30 = 120 + 30 = 150.

O corpo de expressão devolve diretamente o inteiro calculado. Não precisamos de escrever return nesta forma.

Esta operação consulta propriedades e devolve um resultado. Não altera o objeto e não imprime dentro do método.

Com horas entre 0 e 23 e minutos entre 0 e 59, o resultado pertence ao intervalo de 0 a 1439.

O que uma classe deve tornar explícito

Que objetos representa? Livro descreve um exemplar, Hora descreve uma hora do relógio e Emprestimo descreve um empréstimo.

Que dados guarda? Os parâmetros com val ou var tornam-se propriedades. Outras propriedades podem surgir no corpo.

Que condições garante? init e require verificam os dados. Métodos controlados devem preservar as regras ao alterar o estado.

Que operações oferece? Métodos como totalMinutos e devolver dizem o que o resto do programa pode pedir ao objeto.

Como se usa? O construtor cria uma instância. O ponto permite consultar propriedades e chamar métodos acessíveis.

Igualdade numa classe comum

CÓDIGO programas/D01_igualdade_class.kt

```
01 class Leitor(val nome: String)
02
03 fun main() {
04     val a = Leitor("Ana")
05     val b = Leitor("Ana")
06     val c = a
07     println(a == b)
08     println(a === b)
09     println(a === c)
10 }
```

SAÍDA

```
false
false
true
```

Programa completo. O mesmo texto em dois objetos permite separar igualdade e identidade.

a e b resultam de duas chamadas ao construtor. Guardam o mesmo nome, mas são dois objetos distintos.

c recebe a referência de a. A atribuição não constrói nem copia um Leitor.

=== pergunta se as referências designam o mesmo objeto. == chama a operação equals, tratando também a ausência null.

Esta classe não redefine equals. Herda a comparação por identidade de Any, o tipo base comum dos objetos.

data class compara os dados declarados

CÓDIGO programas/D02_igualdade_dados.kt

```
01 data class Leitor(val id: String, val nome: String)
02
03 fun main() {
04     val a = Leitor("R1", "Ana")
05     val b = Leitor("R1", "Ana")
06     println(a == b)
07     println(a === b)
08     println(a)
09 }
```

SAÍDA

```
true
false
Leitor(id=R1, nome=Ana)
```

Programa completo. Acrescentamos data e escolhemos explicitamente os campos que fazem parte do valor.

data modifica a declaração da classe. O compilador gera operações usando as propriedades do construtor primário.

O construtor primário é a lista entre parênteses depois de Leitor. Aqui contém id e nome, ambos com val.

Os campos de a e b coincidem, logo == devolve true. As duas construções continuam a produzir instâncias distintas.

println(a) usa a representação textual gerada. O nome dos campos torna visível o conteúdo do objeto.

As operações que data gera

Neste percurso usamos a geração normal. equals, hashCode e toString também podem ter implementações explícitas.

equals decide a igualdade usada por ==. hashCode devolve um Int para organizar pesquisas em conjuntos e mapas. Objetos iguais têm o mesmo código.

toString produz uma descrição como Leitor(id=R1, nome=Ana). println chama essa operação quando recebe o objeto.

copy constrói outra instância. Permite indicar novos valores para alguns campos e reutiliza os restantes valores do objeto original.

component1, component2, etc. devolvem os campos pela ordem do construtor. São a base da decomposição val (id, nome) = leitor.

A classe precisa de pelo menos um parâmetro no construtor primário, e todos precisam de val ou var. Podemos também acrescentar métodos e init.

copy constrói outro objeto

CÓDIGO programas/D04_copy.kt

```
01 data class Hora(val horas: Int, val minutos: Int)
02
03 fun main() {
04     val original = Hora(8, 5)
05     val seguinte = original.copy(minutos = 6)
06     println(original)
07     println(seguinte)
08     println(original === seguinte)
09 }
```

SAÍDA

```
Hora(horas=8, minutos=5)
Hora(horas=8, minutos=6)
false
```

Programa completo. Esta versão isola copy. Acrescentaremos a validação das horas no exemplo D07.

minutos = 6 é um argumento nomeado: escolhe o parâmetro pelo nome, em vez de o escolher apenas pela posição.

copy usa 6 para minutos e conserva 8 para horas, porque não fornecemos um novo valor para esse campo.

A chamada não atribui nada a original. O objeto original continua a representar 08:05.

seguinte referencia a nova instância. === devolve false porque original e seguinte designam objetos diferentes.

Decomposição pela ordem dos campos

CÓDIGO programas/D05_decomposicao.kt

```
01 data class Hora(val horas: Int, val minutos: Int)
02
03 fun main() {
04     val hora = Hora(8, 5)
05     val (h, m) = hora
06     println(h)
07     println(m)
08     println(hora.component1())
09 }
```

SAÍDA

```
8
5
8
```

Programa completo. A decomposição oferece nomes locais para os componentes de um valor.

val (h, m) = hora declara duas variáveis. Os parênteses agrupam os nomes que receberão os componentes.

h recebe hora.component1(), correspondente a horas. m recebe hora.component2(), correspondente a minutos.

Os nomes h e m são escolhas locais. A ordem dos campos no construtor determina os valores recebidos.

A decomposição consulta o objeto e conserva-o. Não retira os campos nem altera a hora.

Os campos do corpo ficam de fora

CÓDIGO programas/D06_propriedades_corpo.kt

```
01 data class Leitor(val nome: String) {
02     var consultas = 0
03 }
04
05 fun main() {
06     val a = Leitor("Ana")
07     a.consultas = 3
08     val b = a.copy()
09     println(a == b)
10     println(b.consultas)
11     println(a)
12 }
```

SAÍDA

```
true
0
Leitor(nome=Ana)
```

Programa completo. O estado de uma instância pode incluir propriedades que data não considera.

nome aparece no construtor primário. consultas aparece no corpo, entre chavetas, com valor inicial zero.

A igualdade gerada compara apenas nome. Por isso a == b é true apesar de a ter três consultas e b ter zero.

copy não copia o valor atual de consultas. A construção de b executa novamente a inicialização consultas = 0.

toString também omite consultas. A posição de uma propriedade na declaração influencia as operações geradas.

Dados válidos com comportamento

CÓDIGO programas/D07_hora_validada.kt

```kotlin
data class Hora(val horas: Int, val minutos: Int) {
    init {
        require(horas in 0..23)
        require(minutos in 0..59)
    }
    fun totalMinutos(): Int = horas * 60 + minutos
}

fun main() {
    val hora = Hora(8, 5)
    println(hora.totalMinutos())
    println(hora.copy(minutos = 6).totalMinutos())
    // hora.copy(horas = 24)
}
```

SAÍDA

```
485
486
```

Programa completo. Conservamos o modelo Hora do projeto de viagens e acrescentamos a regra do seu domínio.

data class admite init e métodos. A validação ocorre durante cada construção, incluindo uma construção feita por copy.

horas in 0..23 testa o intervalo inclusivo. A condição dos minutos usa o mesmo princípio entre zero e 59.

totalMinutos devolve horas * 60 + minutos. Às 08:05 passaram 480 + 5 = 485 minutos desde as 00:00.

A linha comentada é um teste de erro. Se retirar //, copy tenta construir Hora(24, 5) e require recusa o argumento.

Listas: leitura e alteração

CÓDIGO programas/D08_listas.kt

```
01 fun main() {
02     val nomes: MutableList<String> =
03         mutableListOf("Ana")
04     val leitura: List<String> = nomes
05     nomes.add("Rui")
06     println(leitura)
07     println(nomes === leitura)
08 }
```

SAÍDA

```
[Ana, Rui]
true
```

MutableList<String> significa lista alterável cujos elementos são textos. <String> escolhe o tipo dos elementos e não é uma comparação.

mutableListOf("Ana") constrói uma lista com um elemento. add("Rui") acrescenta outro elemento ao mesmo objeto.

List<String> oferece operações de leitura. leitura recebe a mesma lista, mas esse tipo não disponibiliza add.

A alteração feita através de nomes aparece em leitura. Um acesso apenas de leitura não garante que ninguém possa alterar a lista.

val impede substituir a referência guardada. Não impede as operações de alteração que MutableList oferece.

copy mantém referências aos componentes

CÓDIGO programas/D09_copia_rasa.kt

```
01 data class Pasta(val nomes: MutableList<String>)
02
03 fun main() {
04     val a = Pasta(mutableListOf("Ana"))
05     val b = a.copy()
06     b.nomes.add("Rui")
07     println(a.nomes)
08     println(a === b)
09     println(a.nomes === b.nomes)
10 }
```

SAÍDA

```
[Ana, Rui]
false
true
```

Programa completo. A saída mostra exatamente qual é o objeto novo e qual é o objeto partilhado.

A propriedade nomes guarda uma referência para uma lista. copy constrói outra Pasta e reutiliza essa referência.

Existem duas pastas, mas apenas uma lista. b.nomes.add("Rui") modifica a lista que a também permite consultar.

a === b é false: os objetos exteriores diferem. a.nomes === b.nomes é true: o componente é partilhado.

Esta operação chama-se cópia rasa. copy não percorre os componentes para criar cópias independentes de todos os objetos.

Uma lista separada para a nova pasta

CÓDIGO programas/D10_lista_separada.kt

```
01 data class Pasta(val nomes: MutableList<String>)
02
03 fun main() {
04     val a = Pasta(mutableListOf("Ana"))
05     val outros = a.nomes.toMutableList()
06     val b = a.copy(nomes = outros)
07     b.nomes.add("Rui")
08     println(a.nomes)
09     println(b.nomes)
10     println(a.nomes === b.nomes)
11 }
```

SAÍDA

```
[Ana]
[Ana, Rui]
false
```

Programa completo. A separação da lista é uma decisão explícita de quem constrói a cópia.

toMutableList() cria outra lista com os mesmos elementos. Passamos essa nova lista ao parâmetro nomes de copy.

b guarda agora uma referência diferente da referência guardada em a. Acrescentar Rui em b não altera a lista de a.

Os elementos continuam a ser reutilizados. Aqui são String, que não oferece operações que alterem o próprio texto.

Se os elementos fossem objetos mutáveis, as duas listas ainda poderiam partilhá-los. Copiar a lista só separa esse nível da estrutura.

A igualdade precisa do significado dos dados

CÓDIGO programas/D11_significado_igualdade.kt

```kotlin
data class Fracao(val n: Int, val d: Int)

fun main() {
    val a = Fracao(1, 2)
    val b = Fracao(2, 4)
    println(a == b)
    println(a)
    println(b)
}
```

SAÍDA

```
false
Fracao(n=1, d=2)
Fracao(n=2, d=4)
```

Esta Fracao guarda n e d sem validar nem simplificar. A igualdade gerada compara esses dois inteiros separadamente.

Matematicamente, 1/2 e 2/4 representam o mesmo racional. Os campos guardados diferem, por isso o programa apresenta false.

Para obter igualdade de racionais, podemos normalizar a representação. No projeto de frações, a construção já faz essa normalização.

O mesmo cuidado vale para exemplares: o título não identifica uma cópia física. A escolha dos campos deve refletir o domínio.

data class pode ter estado mutável

CÓDIGO programas/D12_dados_mutaveis.kt

```kotlin
data class Leitura(var valor: Int)

fun main() {
    val a = Leitura(10)
    val b = Leitura(10)
    println(a == b)
    a.valor = 11
    println(a == b)
    println(a.valor)
}
```

SAÍDA

```
true
false
11
```

Programa completo. O tipo da declaração e a possibilidade de alterar o estado são decisões distintas.

var permite atribuir outro valor à propriedade. O modificador data não elimina essa possibilidade.

Inicialmente, a e b têm o mesmo valor. Depois da atribuição a.valor = 11, a igualdade compara 11 com 10.

val a conserva a referência de a. A atribuição à propriedade modifica o objeto que essa referência designa.

Para valores estáveis, prefira propriedades val e componentes sem alterações observáveis. A lista partilhada mostrou por que apenas val pode ser insuficiente.

Playground: segundos de uma duração

EXERCÍCIO exercicios/D13_duracao_exercicio.kt

```kotlin
data class Duracao(val minutos: Int) {
    init { require(minutos in 0..1440) }
    fun segundos(): Int = TODO("Converter")
}

fun main() {
    val a = Duracao(50)
    val b = a.copy(minutos = 60)
    println(a.segundos())
    println(b.segundos())
    println(a.minutos)
    println(a == Duracao(50))
}
```

SAÍDA DEPOIS DE COMPLETAR

```
3000
3600
50
true
```

Exercício completo com TODO. A saída indicada corresponde ao programa depois de completar o método.

Copie o programa completo. Substitua apenas TODO por uma expressão que converta os minutos em segundos.

TODO("...") é uma função que marca trabalho pendente. O programa compila, mas essa chamada lança NotImplementedError quando executada.

Conserve init: o intervalo permitido vai de zero a 1440 minutos. Use a propriedade da instância para calcular a resposta.

Antes de executar, justifique os quatro resultados esperados. Explique por que copy não altera os 50 minutos de a.

Resolução: um método por instância

RESOLUÇÃO resolucoes/D14_duracao_resolucao.kt

```kotlin
data class Duracao(val minutos: Int) {
    init { require(minutos in 0..1440) }
    fun segundos(): Int = minutos * 60
}

fun main() {
    val a = Duracao(50)
    val b = a.copy(minutos = 60)
    println(a.segundos())
    println(b.segundos())
    println(a.minutos)
    println(a == Duracao(50))
}
```

SAÍDA

```
3000
3600
50
true
```

Solução completa. Verifique também Duracao(0) e Duracao(1440): devem produzir 0 e 86400 segundos.

Cada minuto corresponde a 60 segundos. segundos() devolve minutos * 60, usando os minutos do objeto que recebeu a chamada.

a.segundos() usa 50 e devolve 3000. b.segundos() usa 60 e devolve 3600.

copy constrói b com o argumento nomeado minutos = 60. A instância a conserva o campo minutos igual a 50.

A última comparação usa os dados. a e Duracao(50) são instâncias distintas com o mesmo número de minutos.

enum class fixa as alternativas

CÓDIGO programas/E01_enum_valores.kt

```
01 enum class DiaChegada { MESMO, SEGUINTE }
02
03 fun anunciar(dia: DiaChegada) {
04     println(dia)
05 }
06
07 fun main() {
08     val dia: DiaChegada = DiaChegada.SEGUINTE
09     anunciar(dia)
10     println(dia == DiaChegada.MESMO)
11 }
```

SAÍDA

```
SEGUINTE
false
```

Programa completo. As constantes são valores do tipo DiaChegada. Não construímos novas opções com DiaChegada().

DiaChegada é o tipo. MESMO e SEGUINTE são as duas constantes declaradas, separadas por uma vírgula.

DiaChegada.SEGUINTE seleciona uma dessas instâncias. Usamos o nome do tipo seguido de ponto e do nome da constante.

O parâmetro recebe DiaChegada, não um texto livre. Uma String como "amanhã" não pode substituir esse argumento.

A enumeração fixa as opções do dia relativo. Cada viagem continuará a guardar os seus próprios horários e a sua duração.

Propriedades e métodos das constantes

CÓDIGO programas/E02_enum_propriedades.kt

```
01 enum class DiaChegada(val dias: Int, val rotulo: String) {
02     MESMO(0, "mesmo dia"),
03     SEGUINTE(1, "dia seguinte");
04
05     fun minutosAdicionais(): Int = dias * 1440
06 }
07
08 fun main() {
09     val dia = DiaChegada.SEGUINTE
10     println(dia.rotulo)
11     println(dia.minutosAdicionais())
12 }
```

SAÍDA

```
dia seguinte
1440
```

Programa completo. Cada constante já nasce com os valores especificados na declaração.

O construtor da enumeração declara dias e rotulo. MESMO(0, "mesmo dia") fornece os valores para a primeira constante.

O ponto e vírgula termina a lista de constantes antes da declaração do método. As vírgulas separam constantes e argumentos.

minutosAdicionais() consulta dias da constante que recebeu a chamada. Um dia corresponde a 24 * 60 = 1440 minutos.

rotulo é a descrição destinada ao utilizador. Podemos mudá-la sem mudar o nome SEGUINTE usado no código.

when produz um resultado para cada caso

CÓDIGO programas/E03_when.kt

```
01  enum class DiaChegada { MESMO, SEGUINTE }
02
03  fun descricao(dia: DiaChegada): String = when (dia) {
04      DiaChegada.MESMO -> "mesmo dia"
05      DiaChegada.SEGUINTE -> "dia seguinte"
06  }
07
08  fun main() {
09      println(descricao(DiaChegada.MESMO))
10      println(descricao(DiaChegada.SEGUINTE))
11  }
```

SAÍDA

```
mesmo dia
dia seguinte
```

Programa completo. O when é uma expressão: o valor do ramo escolhido torna-se o valor devolvido.

when (dia) escolhe o ramo cujo valor coincide com dia. A seta -> separa a alternativa da expressão produzida.

As duas alternativas devolvem String. O sinal = depois da assinatura torna o when o resultado da função descricao.

Os ramos cobrem todas as constantes, por isso dispensamos else. Esta expressão é exaustiva: qualquer DiaChegada tem resposta.

Se acrescentarmos uma constante, esta função precisará de um ramo correspondente para continuar a cobrir os casos possíveis.

entries e for percorrem as opções

CÓDIGO programas/E04_entries_for.kt

```kotlin
enum class DiaChegada { MESMO, SEGUINTE }

fun main() {
    val dias: List<DiaChegada> = DiaChegada.entries
    println(dias)
    for (dia in dias) {
        println(dia)
    }
}
```

SAÍDA

```
[MESMO, SEGUINTE]
MESMO
SEGUINTE
```

Programa completo. Este exemplo usa entries, disponível em Kotlin 1.9 e versões posteriores.

DiaChegada.entries fornece as constantes pela ordem da declaração. Trata-se de uma lista especializada apenas de leitura.

List<DiaChegada> indica o tipo dos elementos. Os sinais < e > delimitam esse argumento de tipo.

for (dia in dias) executa o corpo uma vez por elemento. Em cada execução, dia designa a constante atual.

O primeiro println apresenta a lista inteira. Os println dentro do ciclo apresentam uma constante por linha.

Nome, posição e código explícito

CÓDIGO programas/E05_name_ordinal.kt

```kotlin
enum class DiaChegada(val codigo: Int) {
    MESMO(10), SEGUINTE(20)
}

fun main() {
    val dia = DiaChegada.SEGUINTE
    println(dia.name)
    println(dia.ordinal)
    println(dia.codigo)
    println(DiaChegada.valueOf("SEGUINTE") == dia)
}
```

SAÍDA

```
SEGUINTE
1
20
true
```

name devolve o nome da constante como String. ordinal devolve a posição na declaração, começando em zero.

O código 20 foi escolhido explicitamente. Ao mudar a ordem das constantes, ordinal pode mudar e codigo pode permanecer igual.

Use uma propriedade própria quando precisar de um identificador estável. A posição da constante serve para conhecer a ordem atual.

valueOf("SEGUINTE") procura o nome exato e devolve a constante. Um nome inexistente lança IllegalArgumentException.

Átomos fixos, regras construídas

CÓDIGO programas/E06_atom_rule.kt

```
01 enum class Atom { A, B, C, D, E }
02 data class Rule(
03     val antecedent: Atom, val conclusion: Atom
04 ) {
05     fun accepts(atom: Atom): Boolean = atom == antecedent
06 }
07
08 fun main() {
09     val regra = Rule(Atom.A, Atom.B)
10     println(regra)
11     println(regra.accepts(Atom.A))
12     println(regra.accepts(Atom.C))
13 }
```

SAÍDA

```
Rule(antecedent=A, conclusion=B)
true
false
```

Programa completo inspirado no projeto de modus ponens. enum class e data class combinam-se no mesmo modelo.

Atom oferece cinco constantes. Rule permite construir regras diferentes usando duas dessas constantes como dados.

antecedent e conclusion têm tipo Atom. A ordem dos argumentos em Rule(Atom.A, Atom.B) define a regra A implica B.

accepts recebe um átomo e compara-o com o antecedente. O resultado Boolean indica se esse átomo corresponde à condição da regra.

Construir ou consultar Rule não demonstra B. Um algoritmo de prova precisará ainda de estabelecer que o antecedente está disponível.

Playground: três dias possíveis

EXERCÍCIO exercicios/E07_enum_exercicio.kt

```kotlin
enum class DiaChegada { MESMO, SEGUINTE, DOIS_DIAS }

fun minutos(dia: DiaChegada): Int = TODO("Usar when")

fun main() {
    for (dia in DiaChegada.entries) {
        println(minutos(dia))
    }
}
```

SAÍDA DEPOIS DE COMPLETAR

```
0
1440
2880
```

Exercício completo com TODO. A saída indicada é o objetivo depois de implementar a função.

Neste exercício ampliamos deliberadamente o exemplo para três opções. O projeto original de viagens continua a usar apenas duas.

Substitua TODO por um when que devolva 0, 1440 ou 2880, consoante o valor recebido.

Escreva um ramo para cada constante e dispense else. O compilador deve verificar que não falta nenhuma alternativa.

O ciclo percorre as constantes pela ordem declarada. Preveja a saída antes de executar o programa completo.

Resolução: todos os casos têm resposta

RESOLUÇÃO resolucoes/E08_enum_resolucao.kt

```kotlin
enum class DiaChegada { MESMO, SEGUINTE, DOIS_DIAS }

fun minutos(dia: DiaChegada): Int = when (dia) {
    DiaChegada.MESMO -> 0
    DiaChegada.SEGUINTE -> 1440
    DiaChegada.DOIS_DIAS -> 2880
}

fun main() {
    for (dia in DiaChegada.entries) {
        println(minutos(dia))
    }
}
```

SAÍDA

```
0
1440
2880
```

Solução completa. Cada constante é um valor fixo, e a função descreve como usar esse valor no cálculo.

O parâmetro dia fornece o valor examinado pelo when. Cada ramo associa uma constante ao número de minutos correspondente.

Os resultados são Int. O corpo de expressão devolve o número escolhido diretamente ao chamador.

O for chama minutos três vezes. A sequência é MESMO, SEGUINTE, DOIS_DIAS, de acordo com entries.

O compilador conhece as alternativas da enumeração. Retirar um ramo torna este when incompleto e impede a compilação.

Uma regra partilhada pelas viagens

CÓDIGO programas/O01_RegrasViagem.kt

```
01  object RegrasViagem {
02      val limite: Int = 1440
03      fun aceita(minutos: Int): Boolean =
04          minutos in 0..limite
05  }
06  fun main() {
07      println(RegrasViagem.limite)
08      println(RegrasViagem.aceita(50))
09      println(RegrasViagem.aceita(1500))
10  }
```

SAÍDA

```
1440
true
false
```

O limite é comum às viagens. Basta uma instância para reunir estas regras.

object RegrasViagem declara um tipo e disponibiliza a sua única instância com esse nome.

limite é uma propriedade do objeto. aceita é um método que devolve Boolean.

RegrasViagem.aceita(50) chama o método diretamente no objeto. Não escrevemos RegrasViagem().

in 0..limite verifica o intervalo inclusivo. A duração de 1500 minutos fica fora dele.

O nome dá acesso à mesma instância

CÓDIGO programas/O02_Referencias.kt

```
01  fun main() {
02      val primeira = RegrasViagem
03      val segunda = RegrasViagem
04      println(primeira === segunda)
05      println(primeira.aceita(50))
06  }
```

SAÍDA

```
true
true
```

Fragmento de main. O ficheiro indicado inclui também RegrasViagem.

O nome RegrasViagem é uma expressão que designa a instância partilhada.

As duas variáveis val recebem referências para essa mesma instância.

=== compara identidade. O resultado true confirma que primeira e segunda designam o mesmo objeto.

A declaração object fica fora de main. A chamada de um método continua a usar ponto e parênteses.

O estado também fica partilhado

CÓDIGO programas/O03_ContadorPartilhado.kt

```
01 object ContadorViagens {
02     var total: Int = 0
03         private set
04     fun registar() {
05         total += 1
06     }
07 }
08 fun main() {
09     val a = ContadorViagens
10     val b = ContadorViagens
11     a.registar()
12     b.registar()
13     println(a.total)
14     println(b.total)
15 }
```

SAÍDA

```
2
2
```

a e b designam o mesmo contador. Ambos os métodos alteram a mesma propriedade total.

total += 1 significa, neste caso, total = total + 1. Cada chamada acrescenta uma unidade.

private set permite ler total, mas reserva a atribuição ao corpo de ContadorViagens.

Este exemplo acompanha duas chamadas numa execução sequencial. Uma nova execução começa novamente em zero.

Uma classe permite contadores independentes

CÓDIGO programas/O04_ContadoresIndependentes.kt

```
01 class Contador {
02     var total: Int = 0
03         private set
04     fun registar() {
05         total += 1
06     }
07 }
08 fun main() {
09     val a = Contador()
10     val b = Contador()
11     a.registar()
12     println(a.total)
13     println(b.total)
14 }
```

SAÍDA

```
1
0
```

Cada chamada Contador() constrói uma instância. A classe descreve o estado de cada uma.

a.registar() modifica apenas o contador designado por a. O contador b conserva zero.

Uma declaração object fornece uma instância comum. Uma classe permite criar as instâncias que forem necessárias.

Para contar empréstimos de bibliotecas independentes, cada biblioteca pode possuir o seu próprio contador.

O lugar de object no projeto

A escolha depende de quantas instâncias o problema precisa.

Uma classe Exemplar permite representar cada cópia física de um livro com os seus próprios dados.

Um object RegrasViagem pode reunir operações comuns que não dependem de uma viagem específica.

Um object com propriedades var partilha essas propriedades por todos os pontos que o usam. Essa partilha faz parte do comportamento.

A inicialização de uma declaração object ocorre no primeiro acesso. A segurança dessa inicialização não torna automaticamente seguras alterações concorrentes ao seu estado.

Playground: conversão com regras comuns

EXERCÍCIO exercicios/O06_ConversorHora.kt

```kotlin
object ConversorHora {
    fun emMinutos(horas: Int, minutos: Int): Int {
        TODO("Validar e converter")
    }
}
fun main() {
    println(ConversorHora.emMinutos(2, 15))
    println(ConversorHora.emMinutos(23, 59))
}
```

SAÍDA DEPOIS DE COMPLETAR

```
135
1439
```

Complete emMinutos. Aceite horas de 0 a 23 e minutos de 0 a 59.

Substitua TODO por duas chamadas require e uma instrução return. Os limites incluem os extremos.

A resposta deve ser horas * 60 + minutos. Cada hora completa acrescenta 60 minutos.

Execute o programa inteiro no Playground. Depois troque uma hora por 24 e observe a recusa.

TODO permite compilar o arranque, mas interrompe a execução quando é chamado. Não é uma resposta válida.

Resolução: validar antes de converter

RESOLUÇÃO resolucoes/O06_ConversorHora.kt

```kotlin
object ConversorHora {
    fun emMinutos(horas: Int, minutos: Int): Int {
        require(horas in 0..23)
        require(minutos in 0..59)
        return horas * 60 + minutos
    }
}
fun main() {
    println(ConversorHora.emMinutos(2, 15))
    println(ConversorHora.emMinutos(23, 59))
}
```

SAÍDA

```
135
1439
```

require testa uma condição. Se for falsa, lança IllegalArgumentException e o método termina com essa falha.

Só chegamos a return depois de ambas as validações terem sucesso.

Para 2 e 15, a expressão calcula 2 * 60 + 15, ou seja, 135.

Não criámos uma instância por conversão. Chamámos duas vezes um método do mesmo ConversorHora.

Uma operação associada à classe

CÓDIGO programas/F01_FabricaDuracao.kt

```
01 class Duracao private constructor(val minutos: Int) {
02     companion object {
03         fun criar(minutos: Int): Duracao {
04             require(minutos in 0..1440)
05             return Duracao(minutos)
06         }
07     }
08 }
09 fun main() {
10     val viagem = Duracao.criar(50)
11     println(viagem.minutos)
12 }
```

SAÍDA

```
50
```

A fábrica criar valida o argumento e devolve uma nova duração.

companion object declara um objeto dentro de Duracao. A classe tem um único companion, partilhado.

Duracao.criar(50) chama criar nesse objeto. Ainda não precisamos de ter uma duração.

private constructor restringe o acesso ao construtor. O companion pode chamá-lo dentro da classe.

viagem guarda a instância devolvida. viagem.minutos consulta a propriedade dessa duração.

Leitura da declaração do construtor

CÓDIGO programas/F01_FabricaDuracao.kt

```
01 class Duracao private constructor(val minutos: Int) {
02     companion object {
03         fun criar(minutos: Int): Duracao {
04             require(minutos in 0..1440)
05             return Duracao(minutos)
06         }
07     }
08 }
```

Fragmento da classe. O ficheiro completo inclui main.

class Duracao introduz o tipo das durações. private constructor explicita a visibilidade do construtor primário.

val minutos: Int cria a propriedade de cada duração. O parâmetro minutos de criar recebe o argumento fornecido na chamada.

O : Duracao depois dos parênteses de criar indica o tipo do resultado.

Em Duracao(minutos), os parênteses chamam o construtor. O return entrega a nova instância ao chamador.

Percurso da chamada Duracao.criar(50)

A chamada só devolve uma duração depois de validar os dados.

1. main chama criar no companion de Duracao e passa o inteiro 50 ao parâmetro minutos.

2. require verifica 50 in 0..1440. A condição é verdadeira e a execução continua.

3. Duracao(minutos) constrói uma instância cuja propriedade minutos contém 50.

4. return entrega essa instância. main associa-lhe o nome viagem e println apresenta 50.

Com 1500, require falha no passo 2. A chamada termina com uma exceção e não devolve uma duração.

O companion e as instâncias são distintos

CÓDIGO programas/F04_InstanciasECompanion.kt

```
01 fun main() {
02     val a = Duracao.criar(50)
03     val b = Duracao.criar(50)
04     val fabrica = Duracao.Companion
05     println(a === b)
06     println(fabrica === Duracao.Companion)
07     println(fabrica.criar(20).minutos)
08 }
```

SAÍDA

```
false
true
20
```

Fragmento de main. O ficheiro indicado inclui a classe Duracao.

Cada execução de criar chama o construtor. a e b são duas durações diferentes, mesmo contendo 50.

Quando omitimos o nome do companion, Kotlin chama-lhe Companion. Duracao.Companion dá acesso ao objeto.

fabrica e Duracao.Companion designam o mesmo objeto. Existe um companion associado à classe.

fabrica.criar(20).minutos encadeia operações: cria a duração e depois consulta minutos no resultado.

Chamadas da fábrica e chamadas da instância

CÓDIGO programas/F05_Chamadas.kt

```
fun main() {
    val d = Duracao.criar(50)
    println(d.minutos)
    // Duracao(50)        // construtor privado
    // Duracao.minutos    // falta uma instância
    // d.criar(50)        // criar está no companion
}
```

SAÍDA

```
50
```

Excerto de main. O ficheiro completo inclui a declaração de Duracao.

Duracao.criar recebe os dados necessários para produzir uma duração. O acesso faz-se pelo nome da classe.

d.minutos lê os dados de uma duração concreta. O acesso faz-se pela referência d.

Uma propriedade de cada instância exige uma instância. A classe não tem um único valor minutos comum a todas.

Os dois caracteres // iniciam um comentário até ao fim da linha. O compilador ignora o texto comentado.

A conta necessária à fábrica de horas

CÓDIGO programas/F06_QuocienteEResto.kt

```
01 fun main() {
02     val total: Int = 95
03     val horas = total / 60
04     val minutos = total % 60
05     println(horas)
06     println(minutos)
07 }
```

SAÍDA

```
1
35
```

Para representar 95 minutos como hora do dia, separamos horas completas e minutos restantes.

/ entre inteiros calcula um quociente inteiro. Para os valores não negativos deste exemplo, 95 / 60 vale 1.

% calcula o resto da divisão. Como 95 = 1 * 60 + 35, 95 % 60 vale 35.

Os resultados são Int. Não surge o número decimal 1,5833… porque ambos os operandos são inteiros.

A fábrica seguinte aceitará apenas 0..1439. Assim, as horas obtidas pertencem a 0..23 e os minutos a 0..59.

Playground: fábrica de uma hora válida

EXERCÍCIO exercicios/F07_HoraDeMinutos.kt

```
01  class Hora private constructor(
02      val horas: Int, val minutos: Int
03  ) {
04      companion object {
05          fun deMinutos(total: Int): Hora {
06              TODO("Validar e construir")
07          }
08      }
09  }
10  fun main() {
11      val h = Hora.deMinutos(95)
12      println(h.horas)
13      println(h.minutos)
14  }
```

SAÍDA DEPOIS DE COMPLETAR

```
1
35
```

Complete deMinutos para aceitar totais de 0 a 1439.

Substitua TODO por require e return. Use total / 60 para as horas e total % 60 para os minutos.

A chamada ao construtor recebe os dois argumentos pela ordem horas, minutos.

Teste também 0, 60 e 1439. As horas esperadas são 0:0, 1:0 e 23:59.

Teste 1440 separadamente: deve ser recusado, pois já corresponde ao início do dia seguinte.

Resolução: a fábrica preserva os limites

RESOLUÇÃO resolucoes/F07_HoraDeMinutos.kt

```
01  class Hora private constructor(
02      val horas: Int, val minutos: Int
03  ) {
04      companion object {
05          fun deMinutos(total: Int): Hora {
06              require(total in 0..1439)
07              return Hora(total / 60, total % 60)
08          }
09      }
10  }
11  fun main() {
12      val h = Hora.deMinutos(95)
13      println(h.horas)
14      println(h.minutos)
15  }
```

SAÍDA

```
1
35
```

O teste total in 0..1439 valida o argumento antes da divisão e da construção.

Para 95, a fábrica calcula os argumentos 1 e 35 e chama Hora(1, 35).

O construtor privado limita a construção ao interior da classe. Os utilizadores chamam deMinutos.

As propriedades val permitem consultar os campos sem lhes atribuir outros valores depois da construção.

Duas responsabilidades no mesmo exemplo

Cada instância representa um valor. O companion oferece uma forma de construir esses valores.

Hora representa uma hora concreta, com propriedades horas e minutos. Várias horas podem coexistir no programa.

Hora.deMinutos é uma fábrica: recebe outra representação, verifica-a e devolve uma instância válida.

O mesmo princípio pode organizar Fracao.criar(numerador, denominador), validando e normalizando antes de devolver a fração.

Uma função de fábrica não tem obrigação de criar sempre uma instância nova. Nos exemplos anteriores, cria, porque chama o construtor em cada execução.

Escolha da construção

As três formas respondem a necessidades diferentes do programa.

class Hora: descreve o tipo e permite criar várias instâncias, cada uma com os seus dados.

object RegrasViagem: disponibiliza uma instância com nome próprio para reunir operações ou estado partilhados.

companion object dentro de Hora: disponibiliza um objeto associado à classe, com membros acessíveis como Hora.deMinutos.

O companion pode existir sem nenhuma instância de Hora. As propriedades horas e minutos continuam a pertencer às instâncias.

O contrato de uma operação

O mesmo programa pode receber diferentes operações.

Uma interface define os membros que um objeto deve disponibilizar para ser usado através desse tipo.

A calculadora precisa de uma operação que receba dois inteiros e devolva um inteiro.

Soma e multiplicação cumprem esse mesmo contrato, embora façam cálculos diferentes.

O compilador verifica os tipos e os membros exigidos. As propriedades matemáticas precisam de justificação e testes.

A assinatura de aplicar

CÓDIGO programas/I01_Classe_Implementa.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
```

Excerto do ficheiro I01. O programa completo aparece a seguir.

interface declara o tipo Operacao. As chavetas delimitam os membros que esse tipo disponibiliza.

fun declara aplicar. a e b são parâmetros, ambos de tipo Int, separados por uma vírgula.

O Int depois dos parênteses indica o tipo do resultado. Cada chamada deve devolver um inteiro.

A declaração termina sem corpo. Uma implementação concreta terá de fornecer o cálculo.

Uma classe que implementa a interface

CÓDIGO programas/I01_Classe_Implementa.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 class Soma : Operacao {
05     override fun aplicar(a: Int, b: Int): Int {
06         return a + b
07     }
08 }
09 fun main() {
10     val soma = Soma()
11     println(soma.aplicar(4, 3))
12 }
```

SAÍDA

```
7
```

class Soma : Operacao declara uma classe que cumpre Operacao. Os dois pontos indicam aqui uma relação entre tipos.

Operacao aparece sem parênteses porque a interface não tem construtor a executar.

override assinala o membro que implementa aplicar. return entrega a soma ao chamador.

Soma() constrói uma instância. soma.aplicar(4, 3) chama o método nessa instância e produz 7.

Polimorfismo: uma função, duas operações

CÓDIGO programas/I02_Polimorfismo.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 class Soma : Operacao {
05     override fun aplicar(a: Int, b: Int): Int = a + b
06 }
07 class Produto : Operacao {
08     override fun aplicar(a: Int, b: Int): Int = a * b
09 }
10 fun mostrar(op: Operacao) {
11     println(op.aplicar(4, 3))
12 }
13 fun main() {
14     mostrar(Soma())
15     mostrar(Produto())
16 }
```

SAÍDA

```
7
12
```

mostrar recebe op com tipo Operacao. Pode receber qualquer instância que implemente essa interface.

O corpo com = devolve diretamente a expressão: a + b na Soma e a * b no Produto.

mostrar(Soma()) chama aplicar na instância de Soma. mostrar(Produto()) chama aplicar na instância de Produto.

mostrar usa a mesma chamada nos dois casos. O objeto recebido determina a implementação executada.

O percurso de uma chamada

Chamada	Objeto recebido	Cálculo	Saída
mostrar(Soma())	Instância de Soma	4 + 3	7
mostrar(Produto())	Instância de Produto	4 * 3	12

Em mostrar(Soma()), primeiro Soma() constrói o objeto. Depois mostrar recebe a referência através do parâmetro op.

op.aplicar(4, 3) executa Soma.aplicar, que devolve 7. println apresenta esse resultado.

Na segunda chamada, op refere uma instância de Produto. Os mesmos argumentos produzem agora 12.

O tipo Operacao determina os membros acessíveis através de op. O objeto determina o corpo que executa.

Um object também implementa uma interface

CÓDIGO programas/I03_Object_Implementa.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 object Soma : Operacao {
05     override fun aplicar(a: Int, b: Int): Int = a + b
06 }
07 object Produto : Operacao {
08     override fun aplicar(a: Int, b: Int): Int = a * b
09 }
10 fun main() {
11     val op: Operacao = Produto
12     println(op.aplicar(4, 3))
13 }
```

SAÍDA

```
12
```

Estas operações não precisam de dados diferentes por instância. object disponibiliza uma única instância de cada operação.

Produto designa diretamente o objeto. Não escrevemos Produto(), porque não estamos a chamar um construtor.

val op: Operacao = Produto guarda a referência usando o tipo da interface.

A chamada op.aplicar(4, 3) executa o método do objeto Produto. O contrato mantém-se igual.

Propriedades e métodos com implementação

CÓDIGO programas/I04_Propriedades_Interface.kt

```
01  interface Rotulavel {
02      val nome: String
03      val rotulo: String
04          get() = "Nome: $nome"
05      fun apresentar() {
06          println(rotulo)
07      }
08  }
09  class Livro(override val nome: String) : Rotulavel
10  fun main() {
11      val livro = Livro("Mensagem")
12      livro.apresentar()
13  }
```

SAÍDA

```
Nome: Mensagem
```

val nome: String exige uma propriedade legível. A classe Livro fornece-a através de override val nome.

rotulo tem um getter: cada acesso calcula a cadeia usando nome. A interface não guarda um campo para rotulo.

apresentar já tem corpo e pode ser usado por Livro sem reescrever o método.

Uma interface pode exigir propriedades e fornecer cálculos. O armazenamento dos dados pertence às implementações.

Um val da interface pode ser implementado por var

CÓDIGO programas/I05_Val_Var_Interface.kt

```
01  interface Nomeado {
02      val nome: String
03  }
04  class Leitor(override var nome: String) : Nomeado
05  fun main() {
06      val leitor = Leitor("Ana")
07      val consulta: Nomeado = leitor
08      println(consulta.nome)
09      leitor.nome = "Rita"
10      println(consulta.nome)
11      // consulta.nome = "Eva"  // Nao compila.
12  }
```

SAÍDA

```
Ana
Rita
```

Nomeado exige a possibilidade de ler nome. Leitor também permite alterá-lo, porque implementa a propriedade com var.

leitor e consulta referem o mesmo objeto. A alteração por leitor torna-se visível quando consulta lê o nome.

Através do tipo Nomeado só existe a leitura prometida por val. A atribuição comentada não compila.

O sentido inverso falha: uma interface que exige var precisa também de escrita, que val sozinho não fornece.

Playground: uma operação de diferença

EXERCÍCIO exercicios/I01_Diferenca.kt

```kotlin
interface Operacao {
    fun aplicar(a: Int, b: Int): Int
}
object Diferenca : Operacao {
    override fun aplicar(a: Int, b: Int): Int {
        TODO("Devolver a menos b")
    }
}
fun main() {
    val op: Operacao = Diferenca
    println(op.aplicar(9, 4))
    println(op.aplicar(4, 9))
}
```

SAÍDA DEPOIS DE COMPLETAR

```
5
-5
```

Saída pretendida depois de completar o método.

Cole o ficheiro indicado sozinho no Playground. Substitua apenas TODO pelo corpo que devolve a menos b.

TODO marca código por completar. Se for executado, lança NotImplementedError e interrompe o programa.

Conserve a interface e a variável op com tipo Operacao. A solução deve funcionar através desse tipo.

Explique por que trocar a ordem dos argumentos muda o sinal. A saída pretendida aparece abaixo.

Resolução: a diferença cumpre o contrato

RESOLUÇÃO resolucoes/I01_Diferenca.kt

```kotlin
interface Operacao {
    fun aplicar(a: Int, b: Int): Int
}
object Diferenca : Operacao {
    override fun aplicar(a: Int, b: Int): Int {
        return a - b
    }
}
fun main() {
    val op: Operacao = Diferenca
    println(op.aplicar(9, 4))
    println(op.aplicar(4, 9))
}
```

SAÍDA

```
5
-5
```

return a - b calcula a diferença e entrega um Int, cumprindo o tipo de resultado exigido.

Na primeira chamada, a recebe 9 e b recebe 4. O resultado é 5.

Na segunda chamada, a recebe 4 e b recebe 9. O resultado é -5.

Também poderíamos escrever override fun aplicar(a: Int, b: Int): Int = a - b.

A calculadora modular completa

Os quatro slides seguintes formam um único programa.

O módulo n fica entre 2 e 12. Cada operando Long será convertido num representante entre 0 e n - 1.

A interface seleciona soma ou multiplicação. A calculadora reduz os operandos, aplica a operação e reduz o resultado.

Com n = 5, os operandos -1 e 3 tornam-se 4 e 3. A soma 7 tem representante 2.

O produto 12 também tem representante 2. Primeiro reduzir evita multiplicar diretamente os operandos Long originais.

Programa completo, parte 1 de 4

CÓDIGO programas/I06_Calculadora_Modular_Completa.kt

```
01 interface Operacao {
02     fun aplicar(a: Int, b: Int): Int
03 }
04 object Soma : Operacao {
05     override fun aplicar(a: Int, b: Int): Int = a + b
06 }
07 object Multiplicacao : Operacao {
08     override fun aplicar(a: Int, b: Int): Int = a * b
09 }
```

Contrato e implementações das duas operações.

Operacao descreve a chamada usada pela calculadora. Os objetos Soma e Multiplicacao fornecem os respetivos cálculos.

a + b e a * b trabalham sobre Int. A calculadora entregará representantes pequenos a estes métodos.

override confirma que cada assinatura corresponde ao membro exigido pela interface.

Cole as quatro partes pela ordem indicada ou use diretamente o ficheiro completo. Existe apenas um main.

Programa completo, parte 2 de 4

CÓDIGO programas/I06_Calculadora_Modular_Completa.kt

```
01 class Modulo(val valor: Int) {
02     init {
03         require(valor in 2..12)
04     }
05     fun representante(x: Long): Int {
06         val n = valor.toLong()
07         return (((x % n) + n) % n).toInt()
08     }
09 }
```

Um módulo válido e a normalização dos operandos.

init executa durante a construção. require exige que valor pertença ao intervalo inclusivo 2..12.

toLong() converte o módulo para Long. Os cálculos com x usam, assim, o mesmo tipo inteiro.

Em Kotlin, % calcula o resto, que pode ser negativo. Somamos n e calculamos outro resto para obter 0..n-1.

toInt() converte o representante final. É seguro porque esse número está entre 0 e 11.

Programa completo, parte 3 de 4

CÓDIGO programas/I06_Calculadora_Modular_Completa.kt

```
01 data class Resultado(
02     val a: Int, val b: Int, val valor: Int
03 )
04 class Calculadora(val modulo: Modulo) {
05     fun calcular(
06         op: Operacao, x: Long, y: Long
07     ): Resultado {
08         val a = modulo.representante(x)
09         val b = modulo.representante(y)
10         val bruto = op.aplicar(a, b)
11         val r = modulo.representante(bruto.toLong())
12         return Resultado(a, b, r)
13     }
14 }
```

Representação do resultado e coordenação do cálculo.

Resultado reúne os representantes usados e o valor final. Cada cálculo devolve uma nova instância dessa data class.

Calculadora recebe e guarda modulo no construtor. calcular recebe uma Operacao e os operandos Long x e y.

As variáveis a e b recebem primeiro os representantes. bruto recebe o resultado de op.aplicar(a, b).

A última normalização produz r. return Resultado(a, b, r) entrega todos os dados ao chamador.

Programa completo, parte 4 de 4

CÓDIGO programas/I06_Calculadora_Modular_Completa.kt

```
01 fun main() {
02     val calc = Calculadora(Modulo(5))
03     val soma = calc.calcular(Soma, -1L, 3L)
04     val produto = calc.calcular(Multiplicacao, -1L, 3L)
05     println("Representantes: ${soma.a} e ${soma.b}")
06     println("Soma: ${soma.valor}")
07     println("Produto: ${produto.valor}")
08 }
```

SAÍDA

```
Representantes: 4 e 3
Soma: 2
Produto: 2
```

Modulo(5) constrói o módulo, que depois entra no construtor de Calculadora.

Soma e Multiplicacao são os objetos passados como operações. O sufixo L identifica literais do tipo Long.

${soma.a} lê uma propriedade dentro de uma expressão interpolada. As chavetas pertencem à interpolação da cadeia.

O programa usa dados fixos e funciona sem entrada pelo teclado. Execute o ficheiro completo no Playground.

Acompanhamento dos valores e dos limites

Operação, módulo 5	Representantes	bruto	r
Soma de -1 e 3	4 e 3	7	2
Produto de -1 e 3	4 e 3	12	2

Para x = -1 e n = 5, a expressão calcula primeiro -1 % 5 = -1, depois 4 % 5 = 4.

Como n é positivo, o primeiro resto está entre -(n - 1) e n - 1. Somar n não excede 23.

Os representantes estão entre 0 e 11. A soma não excede 22 e o produto não excede 121.

Esses limites justificam usar Int nas duas operações. Outra implementação de Operacao exigiria verificar os seus próprios limites.

O que a interface garante

O tipo exprime os membros disponíveis. O problema determina o comportamento esperado.

Qualquer implementação de Operacao tem um método aplicar que recebe dois Int e devolve um Int.

O compilador não demonstra que um objeto chamado Soma soma corretamente. Essa regra exige raciocínio e testes.

Uma implementação pode depender de propriedades internas ou usar outro algoritmo, desde que respeite a assinatura.

A calculadora mantém a normalização dos operandos separada da escolha da operação.

As construções nos projetos da turma

No catálogo, classes controlam os registos e os empréstimos. Tipos de dados representam exemplares e leitores.

Na aritmética modular, uma interface permite escolher operações. Objetos sem estado implementam soma e multiplicação.

Nas frações, uma fábrica no companion pode normalizar valores. Nas viagens, Hora e Duracao distinguem papéis dos inteiros.

No modus ponens, enum representa os átomos e data class representa regras. Cada construção responde a uma necessidade concreta.

Tipos que admitem ausência

CÓDIGO programas/R01.kt

```
01 fun main() {
02     val texto: String? = null
03     val tamanho: Int? = texto?.length
04     val seguro: Int = tamanho ?: 0
05     println(tamanho)
06     println(seguro)
07 }
```

SAÍDA

```
null
0
```

String? admite uma String ou null. String, sem ?, exige um valor não nulo.

null representa a ausência de um valor. Aqui nenhum texto foi fornecido.

?. permite aceder ao membro apenas quando o valor existe. Se texto for null, texto?.length também devolve null.

?: fornece uma alternativa quando o lado esquerdo é null. O resultado de null ?: 0 é 0.

Troque null por "Ana": as saídas passam a 3 e 3. !! força a presença e pode lançar uma exceção. Preferimos tratar a ausência.

Ausência e regresso antecipado

CÓDIGO programas/R02.kt

```kotlin
fun limparNome(texto: String?): String? {
    val recebido = texto ?: return null
    val limpo = recebido.trim()
    if (limpo.isBlank()) return null
    return limpo
}
fun main() {
    println(limparNome(" Ana "))
    println(limparNome("   "))
    println(limparNome(null))
}
```

SAÍDA

```
Ana
null
null
```

A função recebe texto anulável e também pode devolver ausência. Os dois ? fazem parte de tipos distintos.

Se texto for null, ?: return null termina imediatamente esta chamada. Nenhuma linha seguinte da função executa.

Quando existe texto, trim() remove os espaços exteriores. isBlank() verifica se a String está vazia ou só contém espaços.

if decide se a função deve recusar o nome. As chavetas podem ser omitidas quando o ramo contém uma só instrução.

readlnOrNull() pode fornecer uma String? ao ler a consola. Aqui usamos argumentos fixos para reproduzir os três casos no Playground.

Uma recusa pode ser tratada

CÓDIGO programas/R03.kt

```
01 fun main() {
02     try {
03         val hora = 24
04         require(hora in 0..23) { "Hora inválida" }
05         println("Aceite")
06     } catch (erro: IllegalArgumentException) {
07         println(erro.message)
08     }
09     println("Continuamos")
10 }
```

SAÍDA

```
Hora inválida
Continuamos
```

try delimita a operação cuja exceção queremos tratar. catch recebe a exceção do tipo indicado.

require encontra uma condição falsa e lança IllegalArgumentException. A execução salta para o catch correspondente.

A mensagem "Aceite" não aparece: o programa não continua na linha seguinte do bloco try depois da falha.

erro é o nome do parâmetro de catch. erro.message consulta a mensagem. Depois do catch, a execução prossegue.

Trate a falha esperada. Uma mensagem de erro, sozinha, não impede uma alteração de estado: valide antes de alterar.

Uma expressão object cria um objeto anónimo

CÓDIGO programas/R04.kt

```
01 interface Mensagem {
02     fun texto(): String
03 }
04 fun criar(): Mensagem = object : Mensagem {
05     override fun texto(): String = "Olá"
06 }
07 fun main() {
08     val a = criar()
09     val b = criar()
10     println(a.texto())
11     println(a === b)
12 }
```

SAÍDA

```
Olá
false
```

object : Mensagem { ... } é uma expressão. Cria uma implementação de Mensagem sem declarar um nome de classe.

: Mensagem indica a interface implementada. override fornece o corpo do método pedido pela interface.

Cada chamada a criar avalia novamente a expressão object e constrói outra instância.

Por isso a === b é false. A declaração object Regras, usada no módulo anterior, fornece uma instância partilhada.

O tipo de retorno Mensagem expõe as operações da interface. O chamador consegue executar texto() sem conhecer a classe anónima.

Herança de uma classe

CÓDIGO programas/R05.kt

```
01 open class Documento(val titulo: String) {
02     open fun resumo(): String = titulo
03 }
04 class Relatorio(titulo: String) : Documento(titulo) {
05     override fun resumo(): String =
06         "Relatório: ${super.resumo()}"
07 }
08 fun main() {
09     val d: Documento = Relatorio("Biblioteca")
10     println(d.resumo())
11 }
```

SAÍDA

```
Relatório: Biblioteca
```

As classes comuns são fechadas à herança por omissão. open permite criar subclasses. open fun permite redefinir o método.

: Documento(titulo) declara a superclasse e chama o seu construtor com o argumento recebido.

override indica a redefinição de resumo. super.resumo() chama a implementação da superclasse.

A variável d tem tipo Documento e refere um Relatorio. A chamada executa a implementação de Relatorio.

Uma classe pode herdar de uma só classe e implementar várias interfaces. A herança serve quando o novo tipo também é um Documento.

Classe abstrata e implementação concreta

CÓDIGO programas/R06.kt

```
01 abstract class Cartao(val nome: String) {
02     abstract fun descricao(): String
03 }
04 class Leitor(nome: String) : Cartao(nome) {
05     override fun descricao(): String = "Leitor: $nome"
06 }
07 fun main() {
08     val cartao: Cartao = Leitor("Ana")
09     println(cartao.descricao())
10 }
```

SAÍDA

```
Leitor: Ana
```

abstract class declara uma classe que não pode ser instanciada diretamente. Podemos construir as suas subclasses concretas.

A classe abstrata pode receber dados no construtor e guardá-los, como a propriedade nome.

abstract fun declara um método sem corpo. A subclasse concreta tem de fornecer a implementação com override.

Cartao(nome) chama o construtor da superclasse. A propriedade herdada nome está disponível no corpo de Leitor.

Uma interface especifica operações e pode fornecer corpos de métodos, mas não tem campos próprios para guardar o estado de cada instância.

Escolher a construção pelo problema

class: vários objetos com dados e operações. Exemplo: um empréstimo cujo estado muda através de métodos.

data class: valores com igualdade e operações de cópia geradas a partir das propriedades do construtor. Exemplo: uma regra lógica.

enum class: alternativas conhecidas e enumeradas. Exemplo: MESMO_DIA e DIA_SEGUINTE.

object: uma instância partilhada. Exemplo: uma operação de soma sem dados próprios por utilização.

companion object: um objeto associado à classe. Exemplo: uma fábrica que constrói frações normalizadas.

interface: operações comuns a implementações diferentes. Exemplo: aplicar uma operação aritmética escolhida.

Símbolos que mudam de papel

: depois de um nome declara o tipo: nome: String. Depois do nome de uma classe, indica um supertipo: class A : I.

= inicializa, atribui ou introduz um corpo de expressão. == compara igualdade. === compara identidade de objetos.

() delimita parâmetros numa declaração e argumentos numa chamada. {} delimita corpos, lambdas e expressões dentro de ${...}.

<T> indica um argumento ou parâmetro de tipo em contextos como List<String>. < entre números exprime comparação.

? faz parte de um tipo anulável. ?. permite acesso seguro. ?: escolhe uma alternativa na ausência de valor.

-> separa uma condição do seu resultado num when. Numa lambda com parâmetro, separa os parâmetros do corpo.

Documentação e continuação

Sintaxe e funções: kotlinlang.org/docs/basic-syntax.html e kotlinlang.org/docs/functions.html

Classes e propriedades: kotlinlang.org/docs/classes.html e kotlinlang.org/docs/properties.html

Valores e igualdade: kotlinlang.org/docs/data-classes.html e kotlinlang.org/docs/equality.html

Enumerações: kotlinlang.org/docs/enum-classes.html

Objetos e interfaces: kotlinlang.org/docs/object-declarations.html e kotlinlang.org/docs/interfaces.html

Experiências: play.kotlinlang.org. Depois de cada exercício, altere os dados, antecipe a saída e explique o resultado antes de executar.